Assignment No.: 1

HELP University
university of achievers

Assignment Cover Sheet

Student Information (For group assignment, please state names of all members)		**Grade/Marks**
Name	**ID**	
Chan Zi Hao (Team Leader)	B2400169	
Kaung Myat Si Thu	B2401138	
Tan Yen Tze	B2501146	

Module/Subject Information		**Office Acknowledgement**
Module/Subject Code	BIT216	
Module/Subject Name	Software Engineering Principles	
Lecturer/Tutor/Facilitator	Seetha Letchumi	
Due Date	28th September 2026	
Assignment Title/Topic	HELP2U Assignment 1 Task 2	
Intake (where applicable)	Semester 3, 2026	
Word Count		**Date/Time**

Declaration

- I/We have read and understood the Programme Handbook that explains on **plagiarism**, and I/we testify that, unless otherwise acknowledged, the work submitted herein is entirely my/our own.
- I/We declare that no part of this assignment has been written for me/us by any other person(s) except where such collaboration has been authorized by the lecturer concerned.
- I/We authorize the University to test any work submitted by me/us, using text comparison software,for instances of plagiarism. I/We understand this will involve the University or its contractors copying my/our work and storing it on a database to be used in future to test work submitted by others.

Note: 1) The attachment of this statement on any electronically submitted assignments will be deemedto have the same authority as a signed statement.

2) The Group Leader signs the declaration on behalf of all members.

Signature: 	Date:25th September 2026
E-mail: b2400169@helplive.edu.my	

Feedback/Comments*
Main Strengths
Main Weaknesses
Suggestions for improvement

	Student acknowledge feedback/comments
Grader's signature	Student's signature:
Date:	Date:

Note:

1) A soft and hard copy of the assignment shall be submitted.
2) The signed copy of the assignment cover sheet shall be retained by the marker.
3) If the Turnitin report is required, students have to submit it with the assignment. However, departmentsmay allow students up to **THREE** (3) working days after submission of the assignment to submit the Turnitin report. The assignment shall only be marked upon the submission of the Turnitin report.

*Use additional sheets if required.

Table of Contents

1.0 Functional and Non-Functional Requirements 6

1.1 Functional Requirements 6

1.2 Non-Functional Requirements 7

2.0 Requirements Modelling 7

2.1 Use Case 1 7

2.2 Use Case 2 16

2.3 Use Case 3 22

3.0 Design Specifications 29

3.1 Architectural Design Diagram 29

3.2 Sitemap 30

3.3 Entity Relationship Diagram 31

3.4 Database Design and Data Dictionary 32

3.4.1. USER 32

3.4.2. TWOFACTOR_CODE 32

3.4.3. USER_NOTIFICATION 33

3.4.4. VOLUNTEER_PROFILE 34

3.4.5. SKILL 35

3.4.6. AVAILABILITY 35

3.4.7. CATEGORY 36

3.4.8. VOLUNTEER_CATEGORY 36

3.4.9. SUPPORT_REQUEST 36

3.4.10. VOLUNTEER_MATCH 37

3.4.11. SUPPORT_SESSION 38

3.4.12. FEEDBACK 39

3.4.13. Entity Relationships 40

3.5 Wireframes / UX Design 42

4.0 Marking Rubric 52

Table of Figures

Figure 1: HELP2U Architecture Diagram 29
Figure 2: HELP2U Sitemap 30
Figure 3: HELP2U Entity Relationship Diagram 31
Figure 4: Screenshot of the Wireframe: Registration Page 42
Figure 5: Screenshot of the Wireframe: Verification Page 43
Figure 6: Screenshot of the Wireframe: Login Page 43
Figure 7: Screenshot of the Wireframe: Dashboard Page 44
Figure 8: Screenshot of the Wireframe: Profile Page 44
Figure 9: Screenshot of the Wireframe: Volunteer Application Page 45
Figure 10: Screenshot of the Wireframe: Support Request Page 46
Figure 11: Screenshot of the Wireframe: Volunteer Matching Volunteer Page 46
Figure 12: Screenshot of the Wireframe: Selecting Volunteer Page 47
Figure 13: Screenshot of the Wireframe: Incoming Request 48
Figure 14: Screenshot of the Wireframe: Session Time Selection Page 49
Figure 15: Screenshot of the Wireframe: Session Detail Page 49
Figure 16: Screenshot of the Wireframe: History Page 50
Figure 17: Screenshot of the Wireframe: Feedback Page 51

1.0 Functional and Non-Functional Requirements

1.1 Functional Requirements

User Authentication & Profile Management

- Users must be able to manage their student and volunteer profile
- System must support 2-factor authentication using Email OTP
- User must be able to create and edit their profile after authentication
- User must be able to register as a volunteer optionally

Volunteer Registration

- Users that are volunteers must be able to create and edit their information such as:
    - Category
    - Support mode
    - Weekly availability

Support Request & Matching

- All users including students and volunteers must be allowed to make support requests
- Requesters must be able to request for a volunteer from a filtered list that filters by category and support mode
- The requested volunteer must receive a notification to accept or reject the request
- The student must be notified of the volunteer request acceptance or rejection
- If the request is not accepted or rejected within 48 hours, student must be able to request for another volunteer

Session Scheduling

- Upon the acceptance of a request, student must be able to choose their preferred time slot based on the volunteer's weekly availability
- The session schedule must be displayed for both parties to view after time slot is chosen
- Both student and volunteer must be allowed to cancel the session before it starts
- The system must notify the other party when the session is cancelled

Session Completion & Feedback

- The volunteer must be able to mark session as completed
- The student must be able to submit a rating between 1 to 5 with an option to write a comment
- Both student and volunteer must be able to view their history of requests and sessions, including feedback

1.2 Non-Functional Requirements

The system must achieve the following non-functional requirements.

- All pages of the website must be fully loaded under 3 seconds
- All passwords must be hashed before it is stored in the database using Bcrypt hashing algorithm

2.0 Requirements Modelling

2.1 Use Case 1

Student Name and ID: Tan Yen Tze (B2501146)

Epic 1: Student/Volunteer Account and Profile Management

[User Story 1]	As a Student, I need the ability to register for a HELP2U account so that I can access the platform.
Acceptance Criteria	Given the student is not registered yet, When the student enters a valid student ID, name, university email as login name and password submits the registration form,

	Then the system will validate the information, create an account into the system and send a verification code to the respective email address.
MoSCoW	M
Tasks	1. Design UI for registration page 2. Create registration form with fields like student ID, name and university email 3. Setting up an API for registration page 4. Create database to store students' data 5. Implement input validation
Test Cases	[To be filled in assignment 2]
[User Story 2]	As a Student, I need to verify my account so that my account is activated securely.
Acceptance Criteria	Given the student has successfully submitted their registration form and received a verification code, When the student enters the correct verification code, Then the system should verify the code, activate the student account and allow the student to access the platform. Given the student has successfully submitted their registration form and received a verification code,

	When the student enters the wrong verification code for more than 3 times, Then the system will temporarily disable the account from registering for the next 5 mins.
MoSCoW	M
Tasks	1. Design the UI of the account verification page 2. Create input field of the verification code 3. Setting up API for the verification page 4. Connect the verification page to an external authenticator for validation 5. Register the student account into the database when the correct code is entered.
Test Cases	[To be filled in assignment 2]
[User Story 3]	As a Student, I need to login into the website so that I can look for support.
Acceptance Criteria	Given that the student has already registered and the student chooses the login page, When the student is logging in to the website, Then the system should send a verification code to the registered students' email and student can use the given code to authenticate the account and redirect them to the home page.

MoSCoW	M
Tasks	1. Design the login page 2. Create a form with student ID, email and password fields 3. Setting up API for login page 4. Implement login validation 5. Connect the login page to an external authenticator for validation
Test Cases	[To be filled in assignment 2]
[User Story 4]	As a Student, I can reset my password if I have forgotten my password so that I could re-login into the platform.
Acceptance Criteria	Given the student has previously login into the platform successfully and they are on the login page, When the student forgot their password, Then the system will request student to enter a new password and sends a verification code to their registered email to authenticate it and save the new password into the system. Given the email entered is not registered into the platform and they are on the login page,

	When the student clicks on “Forgot Password”, Then the system will show that the account was not registered in the system and prompt student to use a valid email.
MoSCoW	S
Tasks	1. Add a "Forgot Password" option to the login page. 2. Create the password reset request page. 3. Create an input form for the students’ email 4. Setup API for password reset request page 5. Verify that the account exists. 6. Connect the login page to an external authenticator for validation 7. Create the new password page 8. Create form for student to enter new password 9. Save the new password into database
Test Cases	[To be filled in assignment 2]
[User Story 5]	As a Student, I need to update my profile so that my profile is up to date.
Acceptance Criteria	Given the student is registered into the system and the student clicked on “Update” on the profile page,

	When the student makes changes to their profile and save their changes, Then the system should validate and save the information and display it to their profile.
MoSCoW	M
Tasks	1. Design the UI of the student profile page 2. Setting up API for student profile page 3. Display the student's existing profile information. 4. Create form for student to make changes to their current profile
Test Cases	[To be filled in assignment 2]
[User Story 6]	As a Student, I am able to apply myself to become a volunteer so that I can provide help to other students.
Acceptance Criteria	Given the student has a registered account in the system and the student is on the volunteer page, When the student clicks and chooses to be a volunteer, Then it will prompt student to fill in their preferred category, skills, availability and preferred support mode and then the system will save the information into the database and display the “Volunteer” status.
MoSCoW	M

Tasks	1. Design the UI for volunteer registration page. 2. Setting up API for the volunteer registration page 3. Create the volunteer registration form for volunteer skill, availability and preferred mode of assistance 4. Implement data validation
Test Cases	[To be filled in assignment 2]
[User Story 7]	As a Student, I need to link my social media accounts to my HELP2U profile so that I can share my profile to other students.
Acceptance Criteria	Given that the scope of our project is limited to students support and volunteer management function, When the system is being developed, Then the social media integration will not be implemented.
MoSCoW	W
Tasks	-
Test Cases	[To be filled in assignment 2]
[User Story 8]	As a Volunteer, I am able to update my profile so that students can know the latest information about me.

Acceptance Criteria	Given the volunteer has previously registered and it's currently at the volunteer profile page, When the volunteer wants to make changes to their personal skills, interests or preferred support mode, Then the system should save the changes made and display the updated details on their profile.
MoSCoW	M
Tasks	1. Design the UI of the volunteer profile page 2. Setting up API for volunteer profile page 3. Setup tables for volunteer profile page in the database 4. Display the volunteer's existing profile information. 5. Create editable form for volunteer to make changes to their current profile
Test Cases	[To be filled in assignment 2]
[User Story 9]	As a Volunteer, I need to add a short description about myself so that the students can better understand my background.
Acceptance Criteria	Given the volunteer has previously been registered and the volunteer is on the volunteer profile page,

	When the volunteer adds their description profile and saves the changes, Then the system should save the description and display the latest description on the volunteer's profile page.
MoSCoW	C
Tasks	1. Add a profile description field to the volunteer profile page 2. Make an input panel for volunteers to add their description 3. Update the entities for the volunteer profile page
Test Cases	[To be filled in assignment 2]

2.2 Use Case 2

Student Name & ID: Kaung Myat Si Thu (B2401138)

Epic 1: Support Request Creation & Volunteer Matching

User Story 1	As a student, I must create and submit a support request. So that I can get the help I need from a suitable volunteer.
Acceptance Criteria	Given the student is logged in and on the "Request Support" page, When the student selects a support category, enters a description, sets a preferred date, time, and support mode, and clicks "Submit," Then the system validates the information, stores the support request, and displays a confirmation.
MoSCoW	M
Tasks	Design the support request form UI (category, description, date/time, mode fields). Implement client-side and server-side validation for required fields. Create the database table/model to store support requests. Implement the API/controller endpoint to save a new support request.
Test Cases	[To be filled in Assignment 2]
User Story 2	As a student, I need to look for and filter volunteer profiles by skill and available or support mode. So that I can easily locate a volunteer that fits my needs.
Acceptance Criteria	Given the student has submitted a support request, When the student uses the filters in skill, availability, and/or support mode,

	Then the system shows a list of volunteer Profiles that satisfy the selected criteria.
MoSCoW	M
Tasks	Design the volunteer search/filter UI. Implement the query logic to filter volunteers by skill, availability and mode. Display matching volunteer profile cards with key details (skills, availability, mode).
Test Cases	[To be filled in Assignment 2]
User Story 3	As a student, I need the system to give me the best volunteer matches to my request based on my criteria. So that, I can quickly identify the most suitable volunteer without reviewing every profile.
Acceptance Criteria	Given the system has returned a list of volunteers matching the student's filter criteria, When student sees results from search, Then the system then orders or highlights the results by relevance (e.g., skill match, overlap availability, earliest availability). So that, I can quickly identify the most suitable volunteer without reviewing every profile.
MoSCoW	S
Tasks	Define a relevance-scoring rule (e.g. weight by skill match, availability overlap).

	Implement sorting/highlighting of volunteer results by the relevance score. Add UI indicators (e.g. a "Best Match" badge) to top-ranked volunteers.
Test Cases	[To be filled in Assignment 2]

Epic 2: Volunteer Request Review and Response

User Story 1	As a student, I need to select a volunteer and send them my request, so that they can review it.
Acceptance Criteria	Given the student has viewed the list of matching volunteers, When the student chooses a volunteer and clicks "Send", The system then logs a request against the chosen volunteer and informs the volunteer.
MoSCoW	M
Tasks	Implement "select volunteer" action linked to the request record. Trigger a notification (in-app/email) to the selected volunteer on submission. Update the request status field to "Pending".
Test Cases	[To be filled in Assignment 2]
User Story 2	As a volunteer, I want to see the information about a support request I've received.

Acceptance Criteria	Given a volunteer has been informed of a new support request, When the volunteer opens the notification or their request list, Then the system displays the full details of the request.
MoSCoW	M
Tasks	Build the volunteer's incoming-requests list view. Build the request detail view showing all relevant fields. Mark the notification as read once opened.
Test Cases	[To be filled in Assignment 2]
User Story 3	As a volunteer, I must approve/deny an incoming support ticket. So, I can communicate to the student if I can offer the requested support or not.
Acceptance Criteria	Given the volunteer is viewing the details of a pending support request, When the volunteer clicks "Accept" or "Reject", The system then changes the request status to "Accepted" or "Rejected" and informs the student about the request status change.
MoSCoW	M
Tasks	Implement Accept/Reject actions on the request detail view. Update the request status field and timestamp on response. Trigger a status-change notification to the student.
Test Cases	[To be filled in Assignment 2]

User Story 4	As a student I want to be able to have my support request automatically expire if the volunteer does not respond within 48 hrs so that I can pick another volunteer and not waste any more time.
Acceptance Criteria	Given a support request has been sent to a volunteer, When 48 hours pass without the volunteer accepting or rejecting the request. Then, the system indicates that the request has been "Expired", informs the student, and lets the student choose another volunteer for the same request.
MoSCoW	M
Tasks	Create a job/timer to monitor the 48-hour reply time for each request. If there is no reply within the time limit set, update the request status to "Expired". Let the student know that the request has expired and re-enable volunteer selection for that request.
Test Cases	[To be filled in Assignment 2]

Epic 3: Request Status Tracking Notification

User Story 1	As a student, I should be able to see the status of any support request that I have raised. So, I can know if there is any response from a volunteer.
Acceptance Criteria	Given the student has one or more requests for support, When the student clicks on "My Requests", Then the system displays each request along with its current status (Pending/Accepted/Rejected/Expired).

MoSCoW	M
Tasks	Build the "My Requests" list view for students. Query and display request status per record. Add status-based visual indicators (e.g. colour tags).
Test Cases	[To be filled in Assignment 2]
User Story 2	As a student, when the status of my request changes, I need to be notified by email as well as in-app/email notification. So, I can get the notification as soon as possible even if I'm not actively using the app.
Acceptance Criteria	Given a student's support request status has changed (Accepted/Rejected), When a student has an Outlook email verified with their account, Then the system will then send an email summary of the status update as well as an in-app update.
MoSCoW	C
Tasks	Verify student email (Outlook) is confirmed on the student account. Configure Outlook SMTP (or Microsoft Graph API) as the mail transport. Trigger email sending alongside existing notification logic on status change.
Test Cases	[To be filled in Assignment 2]

User Story 3	As a student, I need chat with the volunteer to arrange sessions, preferably live and on the spot, So, I don't have to wait for the status updates.
Acceptance Criteria	Given a support request has been accepted by a volunteer, When the student starts a conversation with the volunteer, Then the student and volunteer can exchange live messages within the platform.
MoSCoW	W
Tasks	(Not planned for this release) Evaluate a real-time messaging/WebSocket solution. (Not planned for this release) Design the chat UI and message data model.
Test Cases	[To be filled in Assignment 2]

2.3 Use Case 3

Student Name & ID: Chan Zi Hao (B2400169)

Epic: Session Scheduling, Management and Feedback

User Story 1	As a student, I need to be able to choose a time slot from the volunteer's available time slots, so that I can confirm my support session schedule.
Acceptance Criteria	Given the support request status of “Accepted”, when the student selects “Schedule Session” and chooses from the available date, time and support mode, then the system will create a session, update the support request status to “Scheduled”, and both the student and volunteer will

	be able to see the session date, time, support mode and participant details.
MoSCoW	M
Tasks	1. Design and develop the UI for picking time slots 2. Create a “support_session” table with foreign key of “volunteer_match”. 3. Write the API endpoints for session scheduling that will validate whether the time slot chosen by the student is within the volunteer’s available time slots. 4. Write the backend logic that will save the session details into the database 5. Develop the calling of the API from the UI.
Test Cases	[To be filled in assignment 2]
User Story 2	As a volunteer, I need to be able to see all my scheduled sessions with details such as date, time, mode and the student's name, so that I can plan ahead of time and show up to all the sessions while keeping up with my studies.
Acceptance Criteria	Given a volunteer has at least one session in "Scheduled" status, when the volunteer logs into the system, then it will show them a list of all their upcoming scheduled session and details such as date, time, mode and the student's name.
MoSCoW	M
Tasks	1. Design and develop the UI for the volunteer’s dashboard to display all upcoming sessions and its details. 2. Write the API endpoint to get session details from the “sessions” table for the logged in volunteer. 3. Develop the frontend logic to call the API to get all upcoming session data to display.

Test Cases	[To be filled in assignment 2]
User Story 3	As a student or volunteer, I need to be able to cancel a scheduled session before the session, so that the other party is able to be notified and does not wait for someone who is not coming.
Acceptance Criteria	Given a session has a "Scheduled" status, when either the student or volunteer selects "Cancel Session", then the system changes the status of the session to "Cancelled" and sends an in-app notification to the other participant.
MoSCoW	M
Tasks	1. Write the API endpoint to update the session status to “Cancelled” 2. Design and develop the UI button to cancel the session. 3. Write the logic to trigger notification to the other participant.
Test Cases	[To be filled in assignment 2]
User Story 4	As a volunteer, I need to mark a support session as completed after it has been done, so that the student can give their feedback and the session is added to my session history.
Acceptance Criteria	Given a session with "Scheduled" status has a scheduled date and time that has passed, when the volunteer clicks "Mark as Completed", then the system changes the status of the session to "Completed" and opens the feedback function to the student. Given a session is not "Scheduled",

	when the volunteer tries to "Mark as Completed", then the system will reject the update and display an error message.
MoSCoW	M
Tasks	1. Build the API endpoint to update status to “Completed” 2. Restrict the endpoint to only the assigned volunteer 3. Reject update of status if status is not in “Scheduled” status 4. Design and develop the UI for volunteer to “Mark as Completed” 5. Write a notification logic that notifies student of completion and wait for their feedback.
Test Cases	[To be filled in assignment 2]
User Story 5	As a student, I need to rate my support session along with an optional choice to write my comment on the session, so that helpful volunteers are appreciated and recommended to others too.
Acceptance Criteria	Given a session with "Completed" status and student has not given their feedback, when the student submits a rating between 1 to 5 with optional comments, then the system saves the feedback and prevents any other feedback for the same session. Given a session is not in "Completed" status, when student tries to interact with the feedback form, then the form input will be disallowed and will state that feedback is only allowed after the completion of the session

MoSCoW	M
Tasks	1. Create “feedback” table with foreign keys from the “support_session” and “user” table 2. Write the API endpoint to insert feedback into the table after the session status and student is validated 3. Design and develop the feedback form with ratings and comments 4. Write the frontend logic to display submitted feedback for both student and volunteer
Test Cases	[To be filled in assignment 2]
User Story 6	As a student or volunteer, I need to easily view my past support requests, sessions and feedback in one page, so that I can look back on my session histories easily.
Acceptance Criteria	Given a user with one or more sessions in "Completed" status, when the user opens their support history page, then the system will show all their past sessions with the session details such as support category, participant name, ratings and comments, if any.
MoSCoW	M
Tasks	1. Write the API endpoint for retrieving all sessions in “Completed” status and its relevant feedback. 2. Design and develop the session history page. 3. Write the frontend logic to call the API for the session history data
Test Cases	[To be filled in assignment 2]
User Story 7	As a student,

	I need to be able to reschedule my session to a different time slot, so that I don't need to cancel my session with the matched volunteer if an emergency occurs.
Acceptance Criteria	Given a session with a "Scheduled" status, when the student clicks "Reschedule" button and pick another time slot from the volunteer's availability, then the system will update the session date and time and notify the volunteer.
MoSCoW	S
Tasks	1. Add a “Reschedule” button 2. Reuse the time slot picker UI for the reschedule UI 3. Add a new API endpoint that allows the session date and time to be updated 4. Write the frontend logic to call the API and update the session date and time
Test Cases	[To be filled in assignment 2]
User Story 8	As a student, I need to add a note describing the contents I want to cover in my support session, so that the volunteer can prepare the necessary materials in advance.
Acceptance Criteria	Given a student is scheduling for a session, when the student types their details in the notes and clicks "Confirm Session", then the system will save the note alongside the session details to be displayed to the volunteer.
MoSCoW	C

Tasks	1. Add a new optional column in the “sessions” table for notes 2. Update the UI to include a text area for notes which is optional 3. Update the frontend UI and logic to display the notes in the volunteer dashboard.
Test Cases	[To be filled in assignment 2]
User Story 9	As a student and volunteer, I need to join a virtual meeting in the platform itself, so that I do not have to arrange for a virtual meeting link myself.
Acceptance Criteria	Given a session is "Scheduled" with a support mode of "Virtual", when the session is about to start, then the system will open a virtual meeting room for both participants to join.
MoSCoW	W
Tasks	Not developing in this project
Test Cases	[To be filled in assignment 2]

3.0 Design Specifications

3.1 Architectural Design Diagram

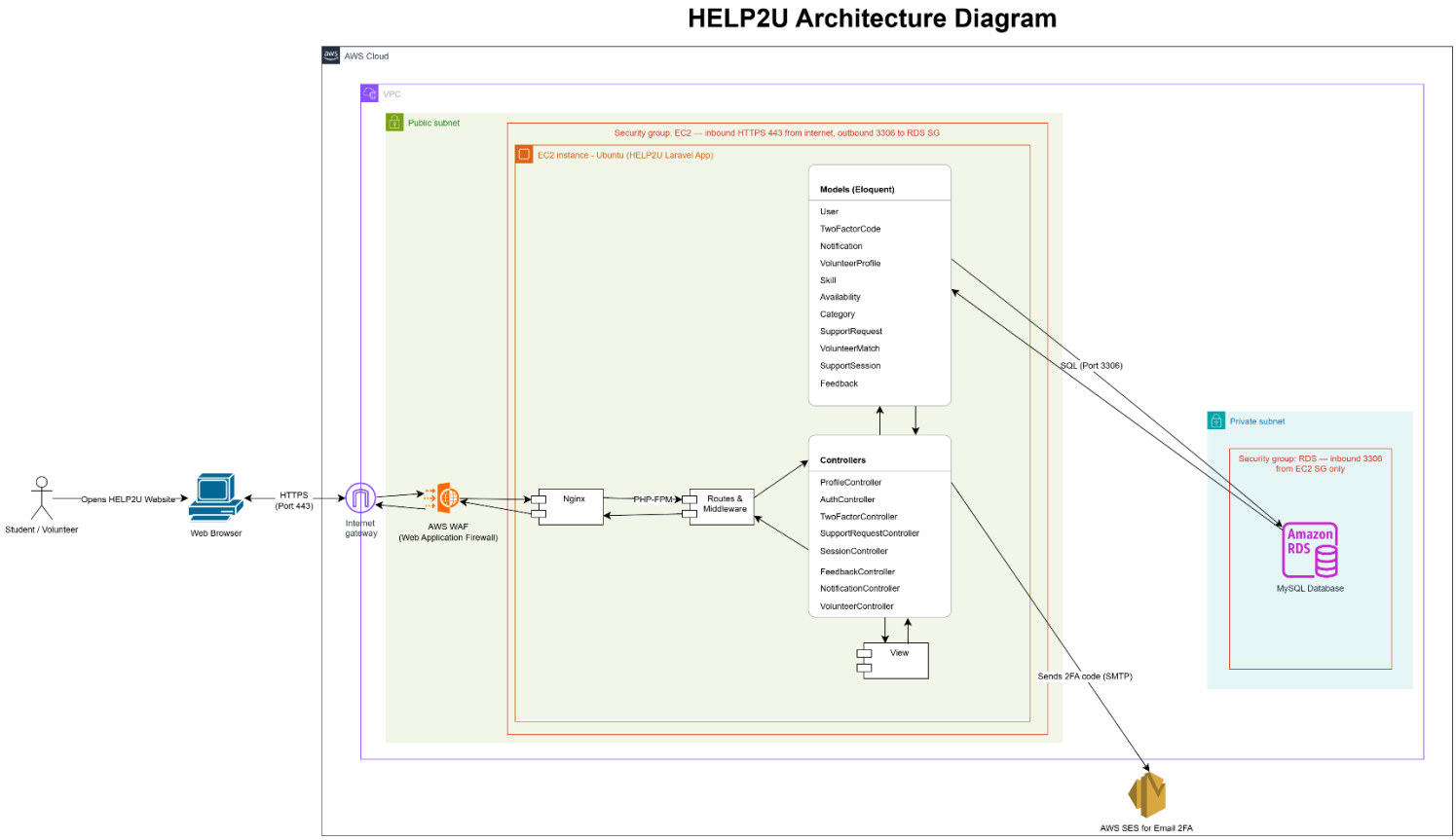


Figure 1: HELP2U Architecture Diagram

3.2 Sitemap

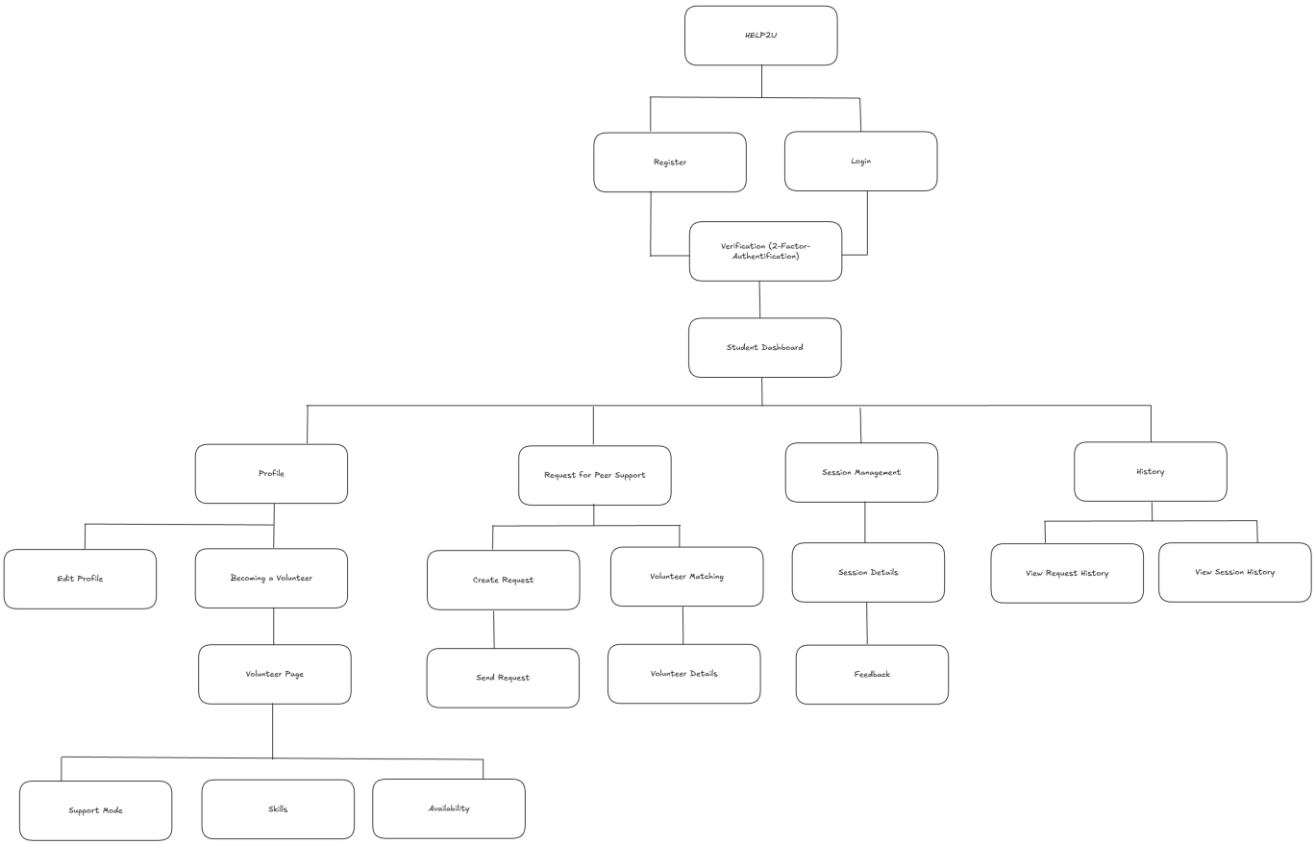


Figure 2: HELP2U Sitemap

3.3 Entity Relationship Diagram

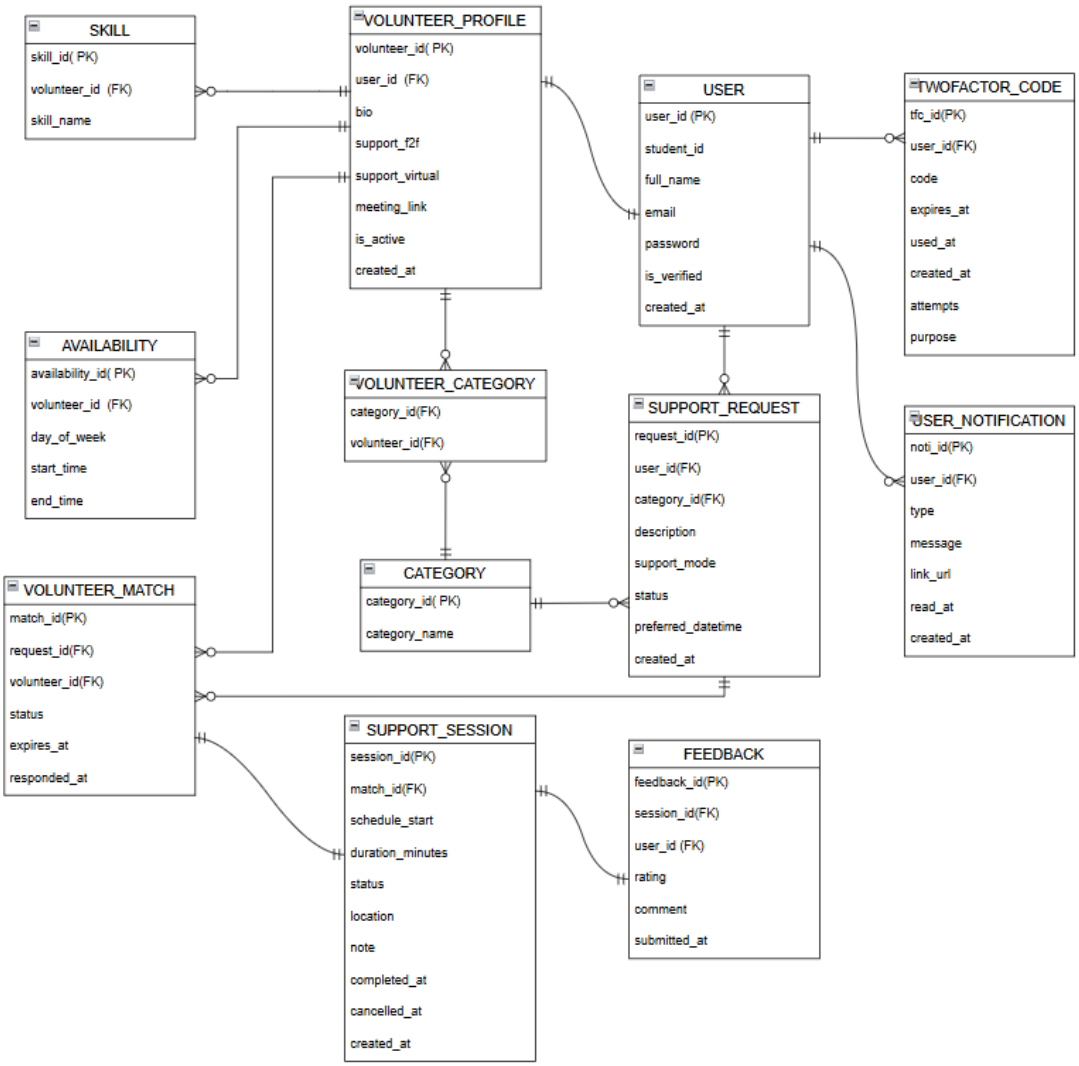


Figure 3: HELP2U Entity Relationship Diagram

3.4 Database Design and Data Dictionary

3.4.1. USER

Field Name	Datatype	Key/Constraint	Description
user_id	INT	PK, Auto-increment	Unique identifier for user account.
student_id	VARCHAR (20)	UNIQUE, NOT NULL	University issued student identification number.
full_name	VARCHAR (100)	NOT NULL	Full name of the user.
email	VARCHAR (100)	UNIQUE, NOT NULL	University email address, especially Outlook mail, used for login (2FA). Eg: B1234567@helplive.edu.my
password	VARCHAR (255)	NOT NULL	Hash password for authentication.
is_verified	BOOLEAN	NOT NULL, Default: FALSE	When the user has completed email/ 2FA verification.
created_at	DATETIME	NOT NULL	Timestamp when account was created.

3.4.2. TWOFACTOR_CODE

Field Name	Datatype	Key/Constraint	Description
tfc_id	INT	PK, Auto-increment	Unique identifier for 2FA Code Record.

user_id	INT	FK, NOT NULL	Foreign Key constraint from USER.user_id.
code	VARCHAR (10)	NOT NULL	The one-time verification code for 2FA.
expires_at	DATETIME	NOT NULL	Time after which the code is no longer valid.
used_at	DATETIME	NULLABLE	Timestamp the code was consumed. If null then, code is unused.
created_at	DATETIME	NOT NULL	Timestamp when the code was generated.
attempts	INT	NOT NULL Default: 0	Number of failed verifications attempts for this code; used to enforce the 3-attempt lockout.
purpose	VARCHAR (50)	NOT NULL	Reason the code was issued. Eg: REGISTER, LOGIN, RESET

3.4.3. USER_NOTIFICATION

Field Name	Datatype	Key/Constraint	Description
noti_id	INT	PK, Auto-increment	Unique identifier for user notification.
user_id	INT	FK, NOT NULL	Foreign Key constraint from USER.user_id.

type	VARCHAR (50)	NOT NULL	Category of Notification. Eg: MATCH, SESSION, SYSTEM
message	VARCHAR (255)	NOT NULL	Notification body test.
link_url	VARCHAR (255)	NULLABLE	Deep link to the related record (request, session, etc.).
read_at	DATETIME	NULLABLE	Timestamp, the user read the notification.
created_at	DATETIME	NOT NULL	Timestamp when the notification was generated.

3.4.4. VOLUNTEER_PROFILE

Field Name	Datatype	Key/Constraint	Description
volunteer_id	INT	PK, Auto-increment	Unique identifier for volunteer profile.
user_id	INT	FK, UNIQUE, NOT NULL	Foreign Key constraint from USER.user_id.
bio	TEXT	NULLABLE	Short self-introduction shown to requesters.
support_f2f	BOOLEAN	NOT NULL, Default: FALSE	When volunteer offer face-to-face support.
support_virtual	BOOLEAN	NOT NULL, Default: FALSE	When volunteer offer virtual/online meeting support.

meeting_link	VARCHAR (255)	NULLABLE	Default video-call link for virtual support.
is_active	BOOLEAN	NOT NULL, Default: TRUE	The volunteer is currently accepting matches or not.
created_at	DATETIME	NOT NULL	Timestamp volunteer profile was created.

3.4.5. SKILL

Field Name	Datatype	Key/Constraint	Description
skill_id	INT	PK, Auto-increment	Unique identifier for skill.
volunteer_id	INT	FK, NOT NULL	Foreign Key constraint from VOLUNTEER_PROFILE. volunteer_id.
skill_name	VARCHAR (100)	NOT NULL	Name of skills. Eg: Calculus, Maths, Programming, Management, etc...

3.4.6. AVAILABILITY

Field Name	Datatype	Key/Constraint	Description
availability_id	INT	PK, Auto-increment	Unique identifier for availability slot.
volunteer_id	INT	FK, NOT NULL	Foreign Key constraint from VOLUNTEER_PROFILE.volunteer_id.

day_of_week	VARCHAR (20)	NOT NULL	Day of the slot that applies to. Eg: Monday, Tuesday
start_time	TIME	NOT NULL	Slot starts time.
end_time	TIME	NOT NULL, > start_time	Slot end time.

3.4.7. CATEGORY

Field Name	Datatype	Key/Constraint	Description
category_id	INT	PK, Auto-increment	Unique identifier for category.
category_name	VARCHAR (100)	UNIQUE, NOT NULL	Display name of the category.

3.4.8. VOLUNTEER_CATEGORY

Field Name	Datatype	Key/Constraint	Description
category_id	INT	PK (composite), FK	Category, the volunteer support.
volunteer_id	INT	PK (composite), FK	Volunteer linked to the category.

3.4.9. SUPPORT_REQUEST

Field Name	Datatype	Key/Constraint	Description
request_id	INT	PK, Auto-increment	Unique identifier for support request.

user_id	INT	FK, NOT NULL	Foreign Key constraint from USER.user_id.
category_id	INT	FK, NOT NULL	Foreign Key constraint from CATEGORY.category_id.
description	TEXT	NOT NULL	Free text description of the support needed.
support_mode	VARCHAR (20)	NOT NULL	Request mode: F2F, VIRTUAL
status	VARCHAR (20)	NOT NULL, Default: PENDING	To save the request status. Eg: PENDING, ACCEPTED, REJECTED, EXPIRED, SCHEDULED, COMPLETED, CANCELLED
preferred_datetime	DATETIME	NULLABLE	Preferred date and time that students would like to make session.
created_at	DATETIME	NOT NULL	Timestamp the request was submitted.

3.4.10. VOLUNTEER_MATCH

Field Name	Datatype	Key/Constraint	Description
match_id	INT	PK, Auto-increment	Unique identifier for match record.
request_id	INT	FK, NOT NULL	Foreign Key constraint from SUPPORT_REQUEST. request_id.
volunteer_id	INT	FK, NOT NULL	Foreign Key constraint from VOLUNTEER_PROFILE.volunteer_id.

status	VARCHAR (20)	NOT NULL, Default: PENDING	PENDING, ACCEPTED, REJECTED or EXPIRED.
expires_at	DATETIME	NULLABLE	Timestamp the match offer expires when volunteer doesn't respond.
responded_at	DATETIME	NULLABLE	Timestamp the volunteer ACCEPT/DENIED.

3.4.11. SUPPORT_SESSION

Field Name	Datatype	Key/Constraint	Description
session_id	INT	PK, Auto-increment	Unique identifier for session.
match_id	INT	FK, NOT NULL, UNIQUE	Foreign Key constraint from VOLUNTEER_MATCH. match_id.
schedule_start	DATETIME	NOT NULL	Planned start date and time of the session.
duration_minutes	INT	NOT NULL	Planned duration of the session in minutes.
status	VARCHAR (20)	NOT NULL, Default:SCHEDULED	SCHEDULED, COMPLETED or CANCELLED
location	VARCHAR (255)	NULLABLE	Meeting location for face-to-face sessions. Null when the session is virtual.
notes	TEXT	NULLABLE	Additional notes or instructions for the session,

			added by the volunteer or student.
completed_at	DATETIME	NULLABLE	Timestamp of the session marked complete.
cancelled_at	DATETIME	NULLABLE	Timestamp of the session was cancelled.
created_at	DATETIME	NOT NULL	Timestamp the session record was created.

3.4.12. FEEDBACK

Field Name	Datatype	Key/Constraint	Description
feedback_id	INT	PK, Auto-increment	Unique identifier for feedback record.
session_id	INT	FK, NOT NULL, UNIQUE	Foreign Key constraint from SUPPORT_SESSION. session_id.
user_id	INT	FK, NOT NULL	Foreign Key constraint from USER.user_id.
rating	INT	NOT NULL, 1-5	Numeric rating given for the session. Between 1 to 5.
comment	VARCHAR (200)	NULLABLE	Free text, user comment.
submitted_at	DATETIME	NOT NULL	Timestamp the feedback was submitted.

3.4.13. Entity Relationships

Parent Entity	Child Entity	Cardinality	Business Rule
USER	VOLUNTEER_PROFILE	1 : 1 (optional)	A user may register one volunteer profile.
USER	SUPPORT_REQUEST	1 : M	A user (student) can submit many support requests.
USER	TWOFACTOR_CODE	1 : M	A user can have many 2FA codes issued over time.
USER	USER_NOTIFICATION	1 : M	A user can receive many notifications.
USER	FEEDBACK	1 : M	A user can submit many feedback records.
VOLUNTEER_PROFILE	SKILL	1 : M	A volunteer can list many skills.
VOLUNTEER_PROFILE	AVAILABILITY	1 : M	A volunteer can define many availability slots.

VOLUNTEER_PROFILE	VOLUNTEER_CATEGORY	1 : M	A volunteer can support many categories (via junction).
CATEGORY	VOLUNTEER_CATEGORY	1 : M	A category can be supported by many volunteers (via junction).
CATEGORY	SUPPORT_REQUEST	1 : M	A category can classify many support requests.
SUPPORT_REQUEST	VOLUNTEER_MATCH	1 : M	A request can generate multiple candidates matches.
VOLUNTEER_PROFILE	VOLUNTEER_MATCH	1 : M	A volunteer can be proposed as a candidate for many matches.
VOLUNTEER_MATCH	SUPPORT_SESSION	1 : 1	An accepted match results in exactly one support session.
SUPPORT_SESSION	FEEDBACK	1 : 1	A completed session receives at

			most one feedback record.

3.5 Wireframes / UX Design

Use Case 1: Manage Student & Volunteer Profile

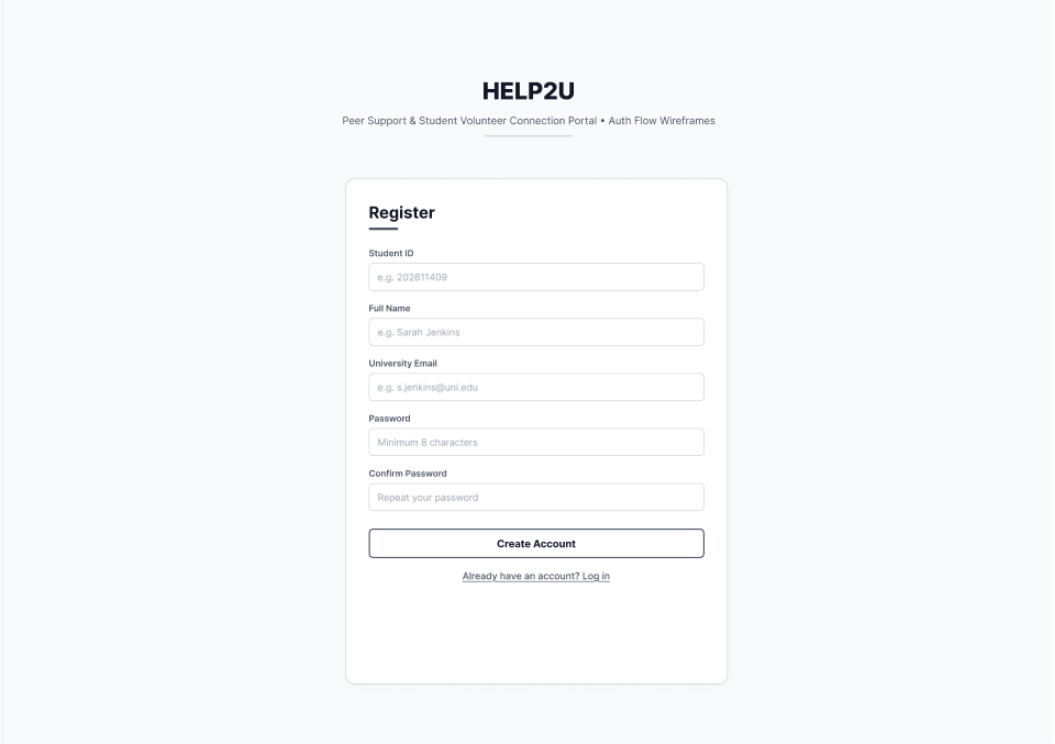


Figure 4: Screenshot of the Wireframe: Registration Page

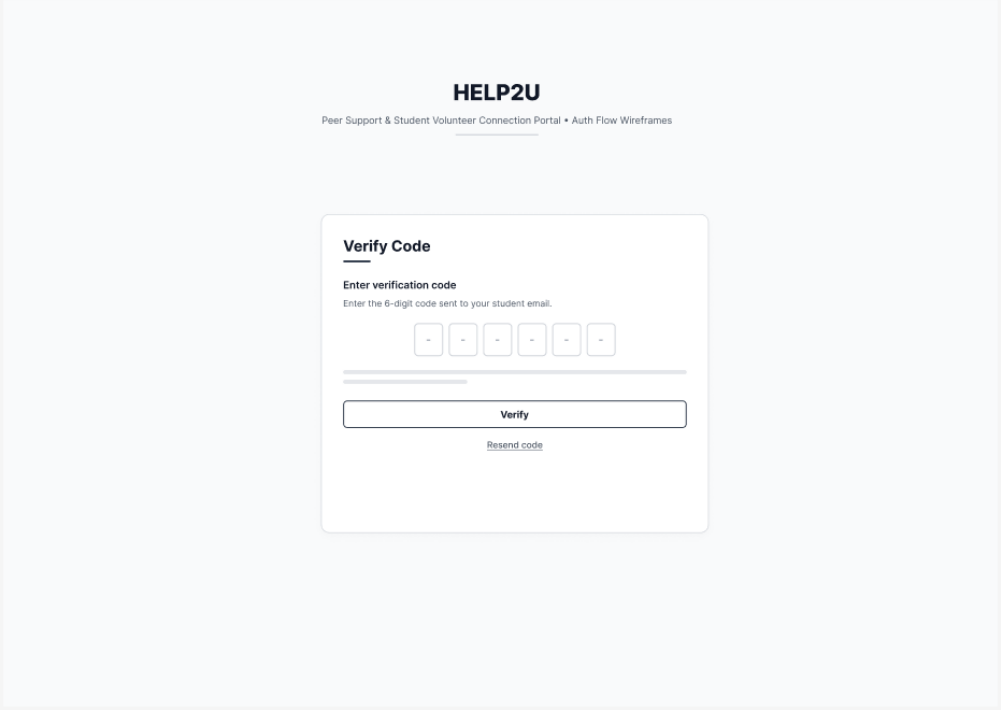


Figure 5: Screenshot of the Wireframe: Verification Page

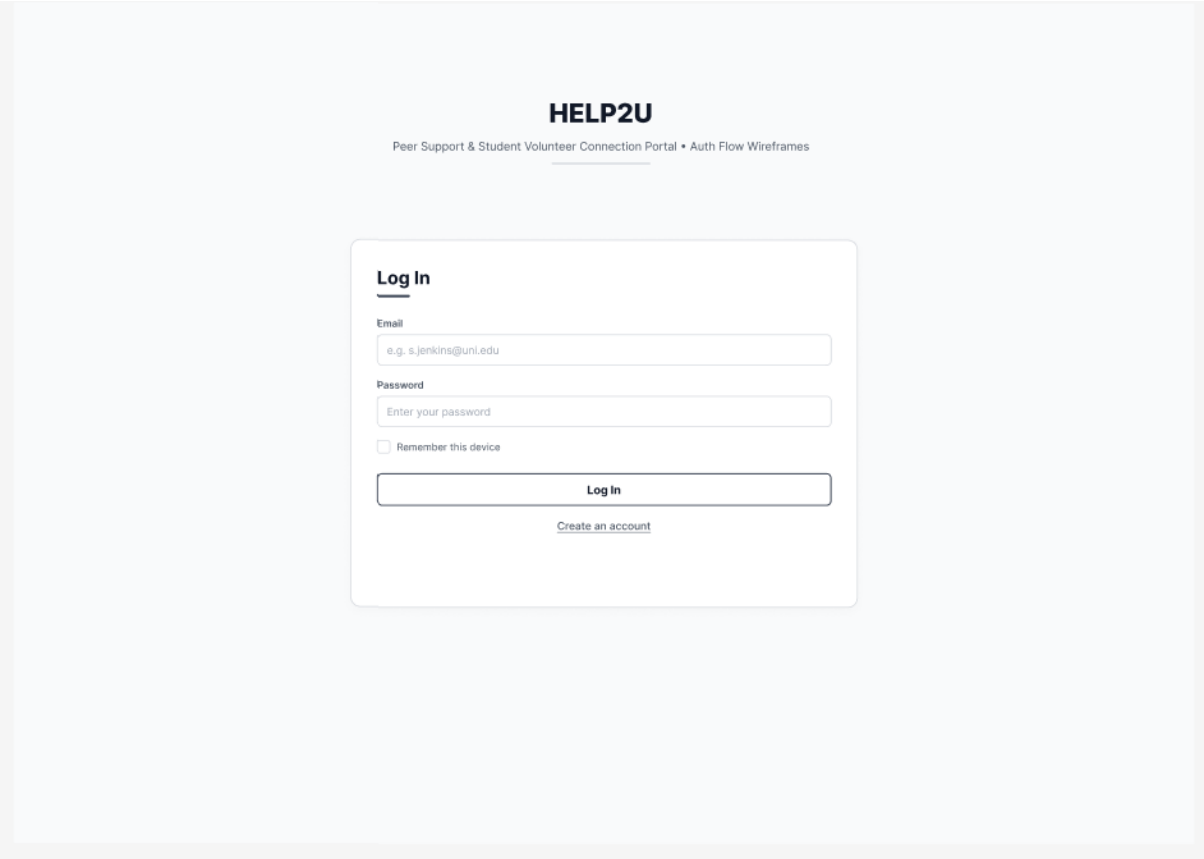


Figure 6: Screenshot of the Wireframe: Login Page

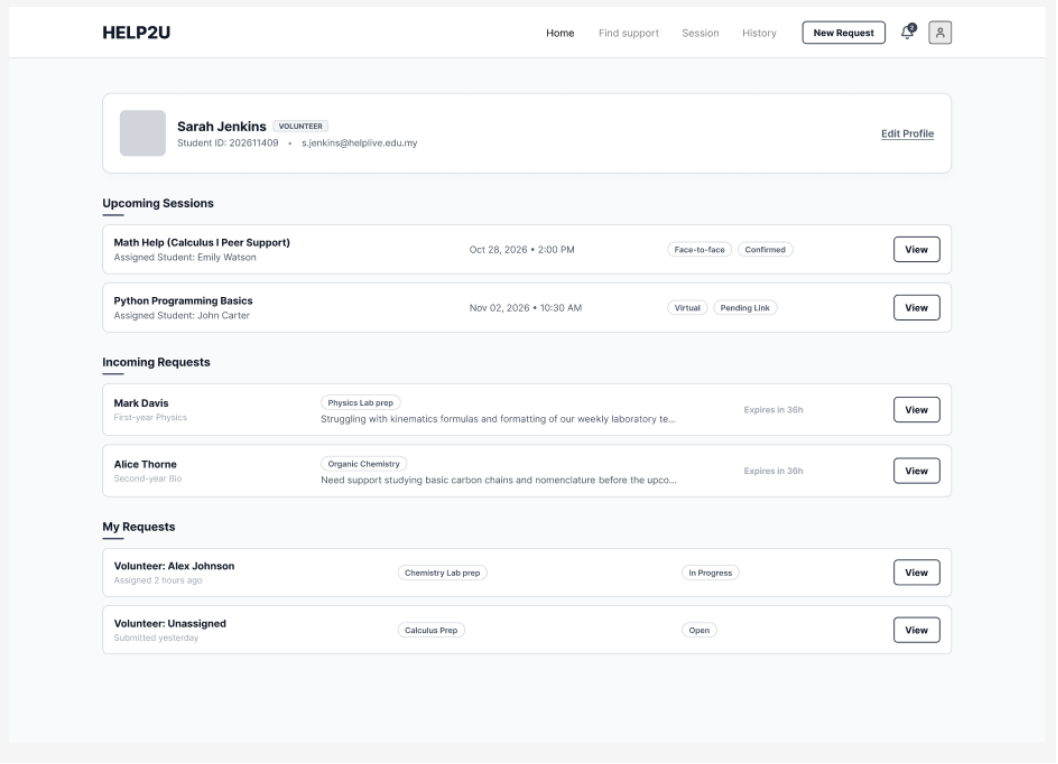


Figure 7: Screenshot of the Wireframe: Dashboard Page

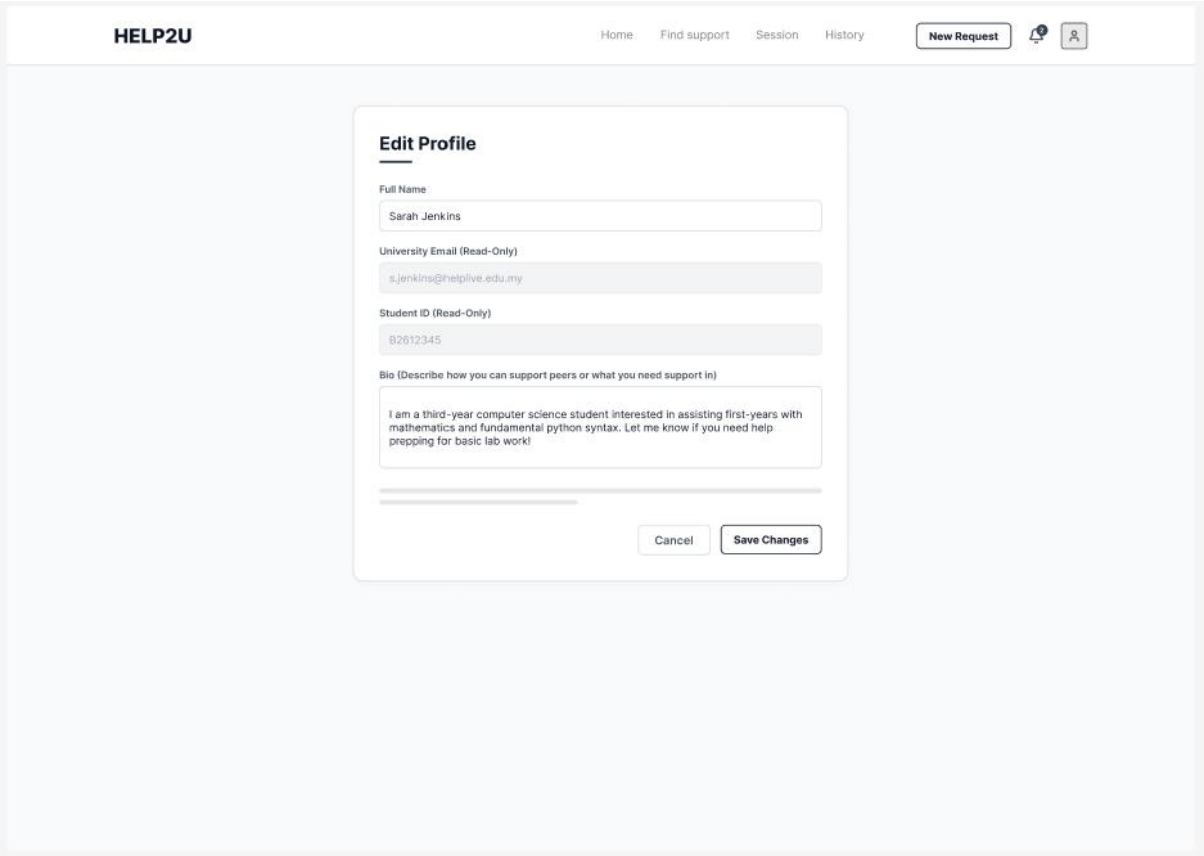


Figure 8: Screenshot of the Wireframe: Profile Page

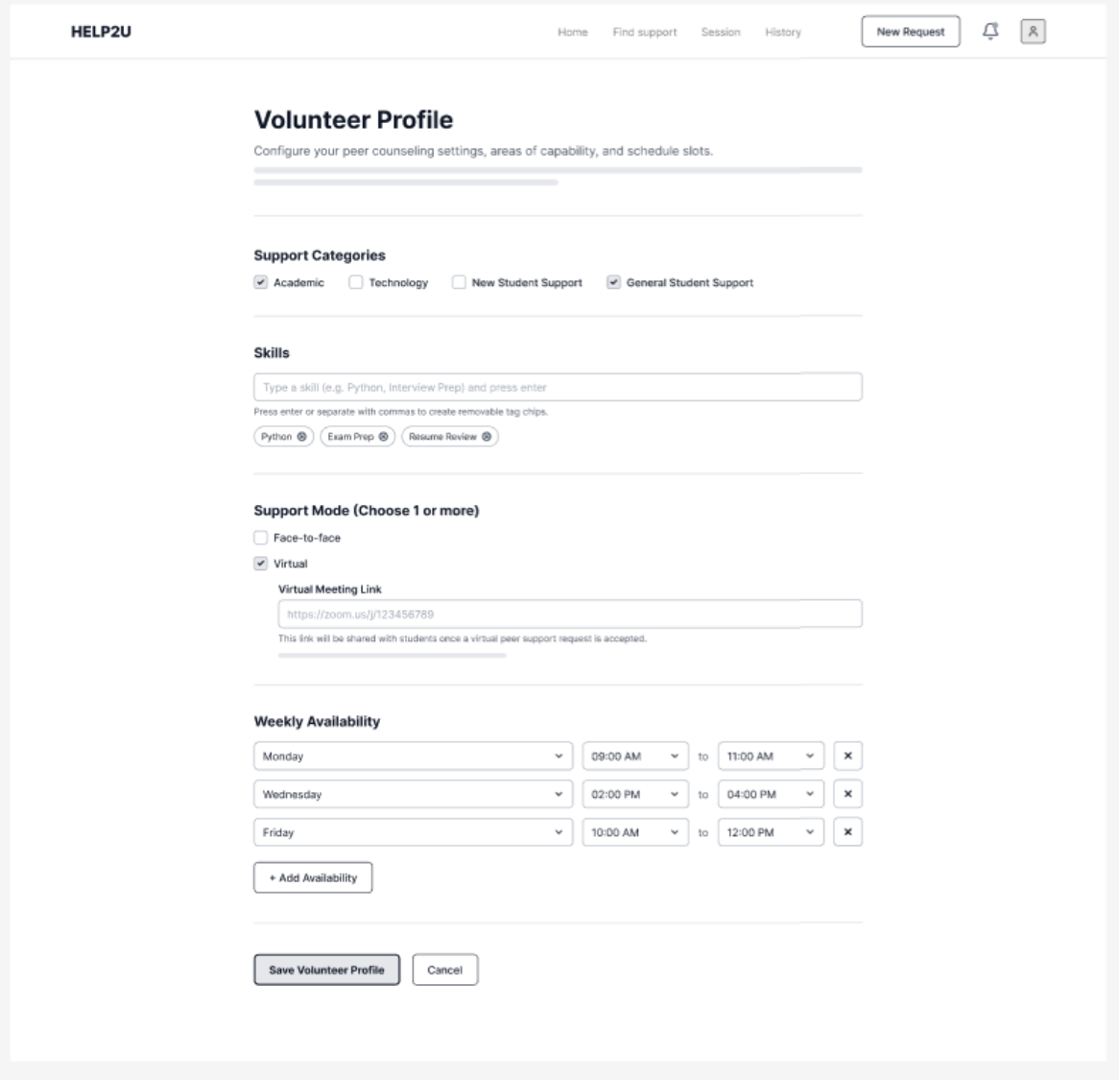


Figure 9: Screenshot of the Wireframe: Volunteer Application Page

Use Case 2: Support Request and Volunteer Matching

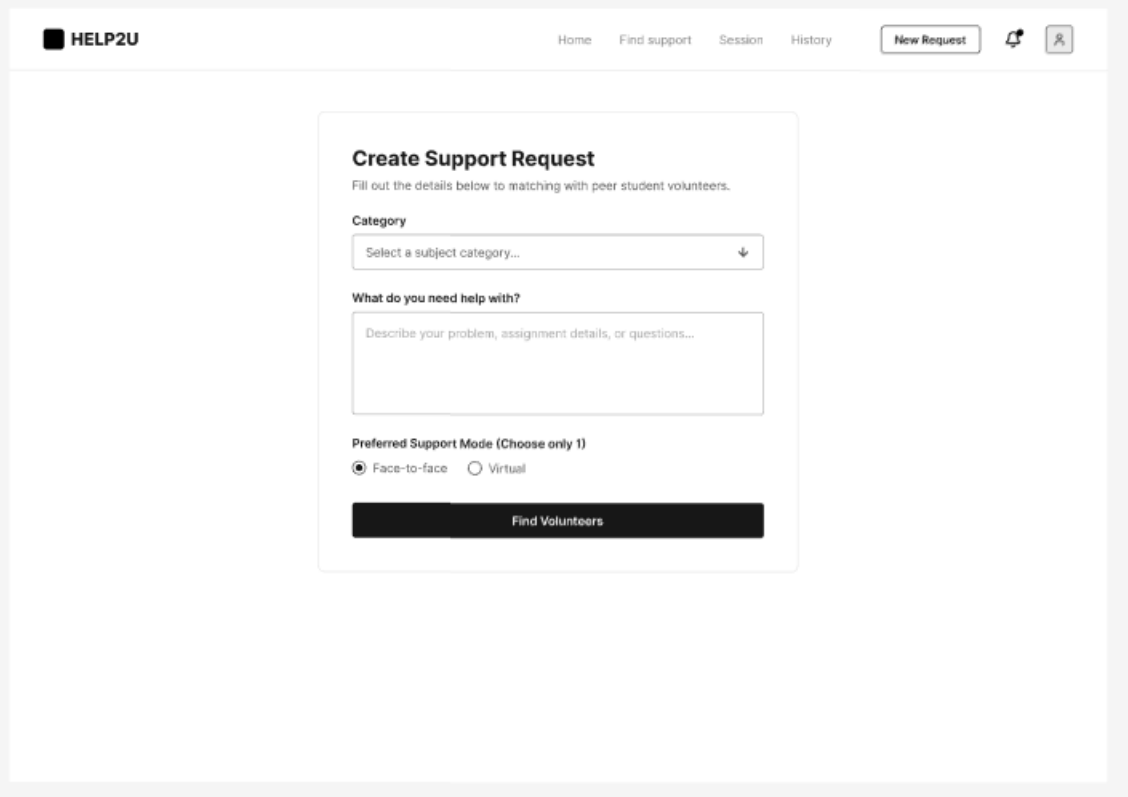


Figure 10: Screenshot of the Wireframe: Support Request Page

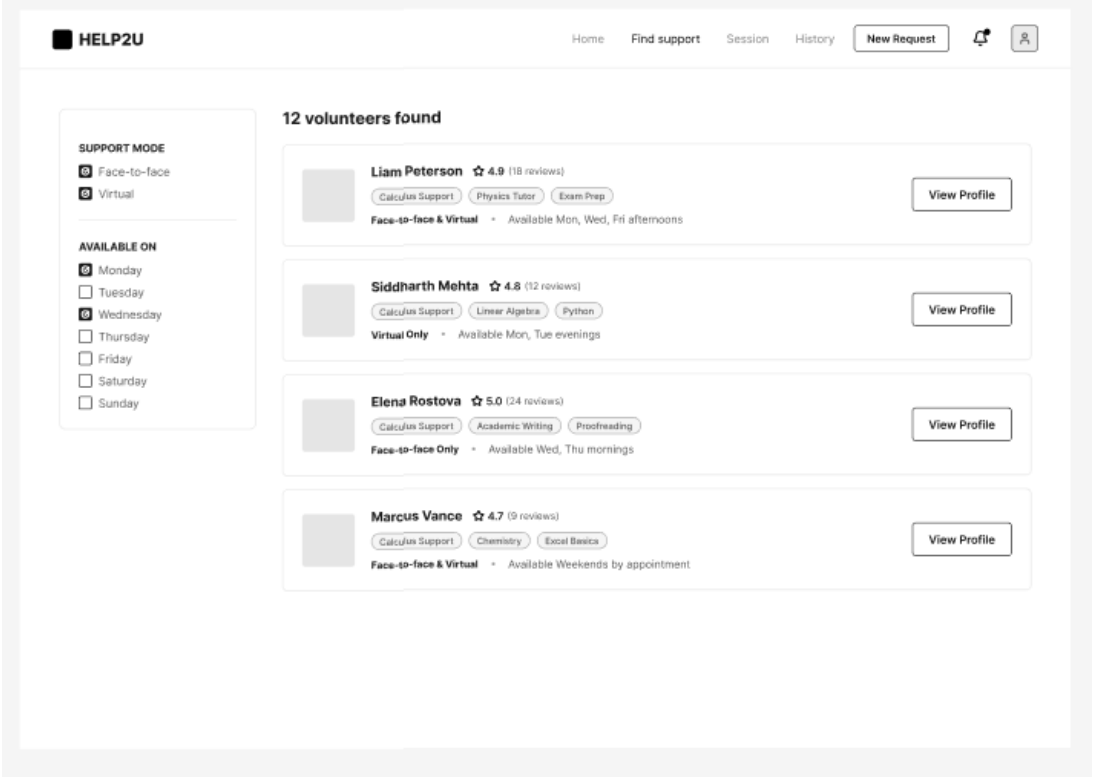


Figure 11: Screenshot of the Wireframe: Volunteer Matching Volunteer Page

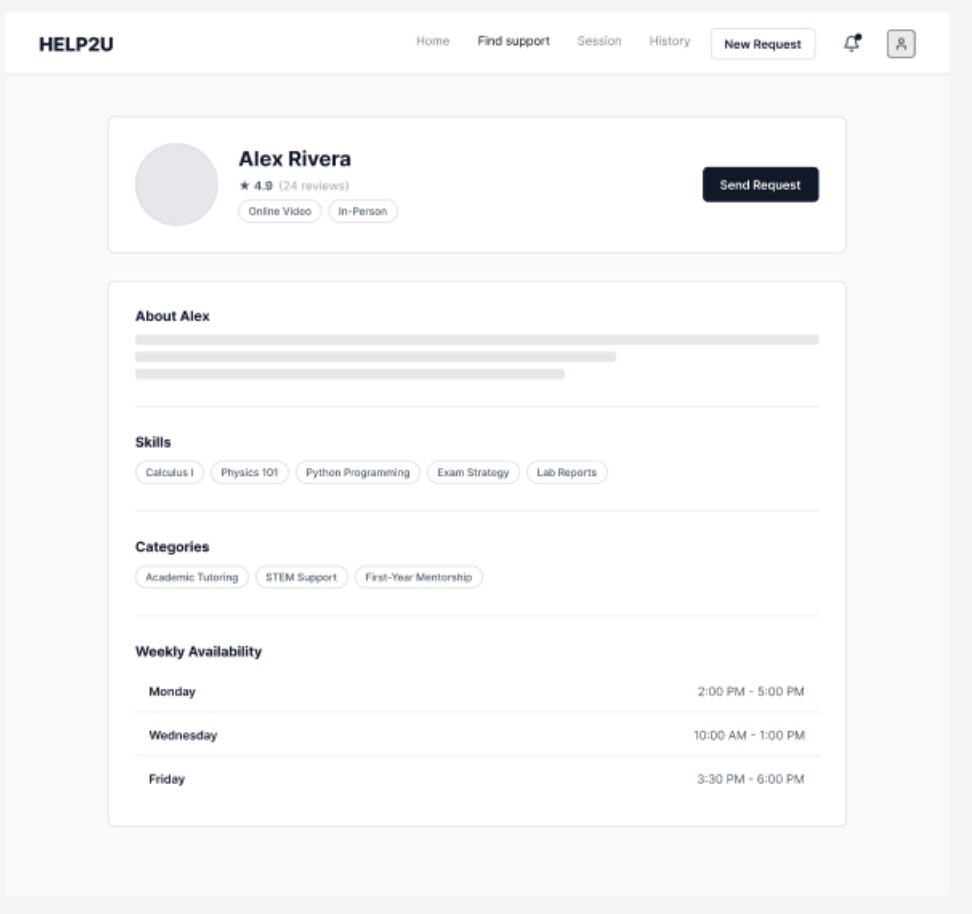


Figure 12: Screenshot of the Wireframe: Selecting Volunteer Page

Use Case 3: Manage Support Session and Feedback

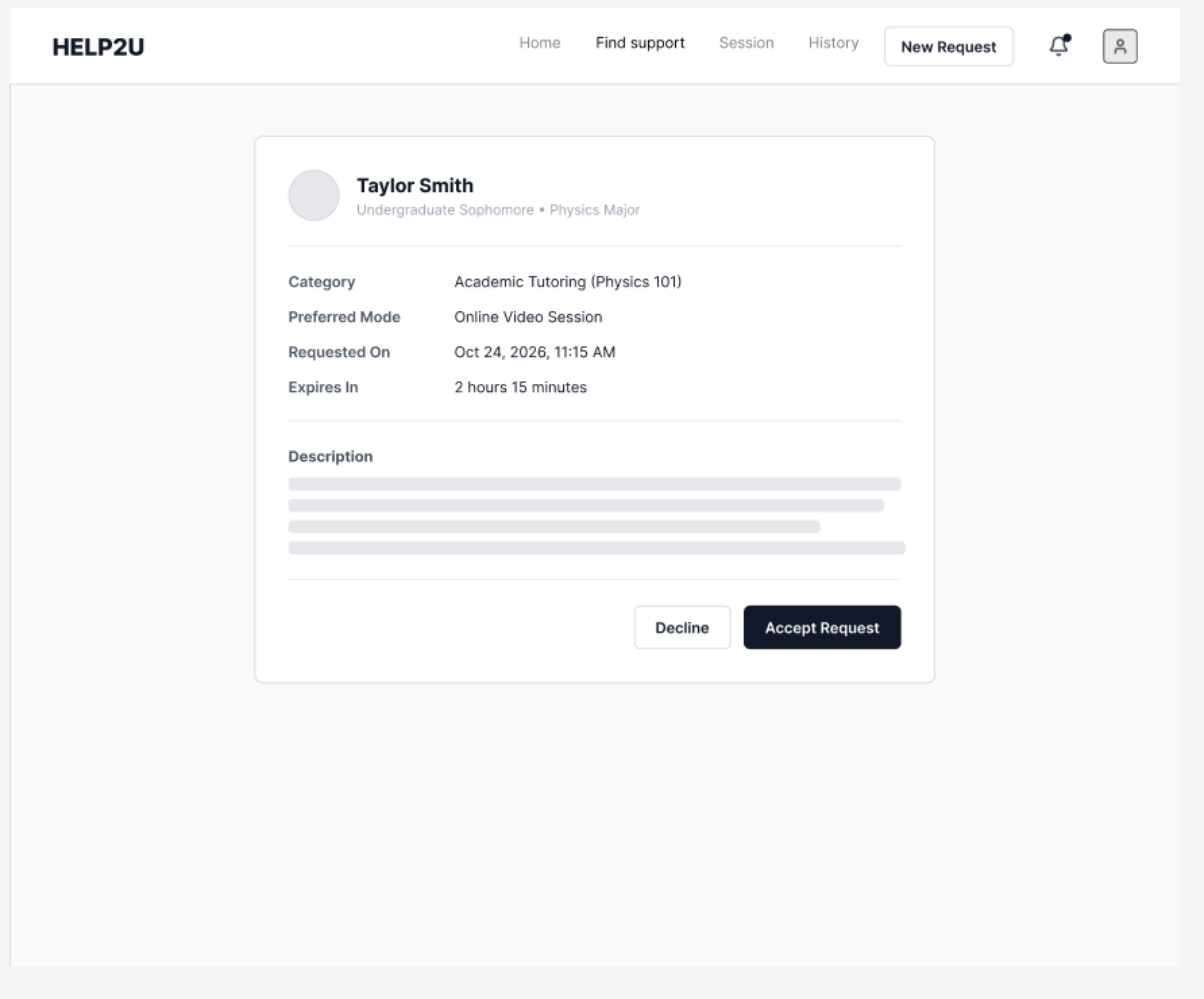


Figure 13: Screenshot of the Wireframe: Incoming Request

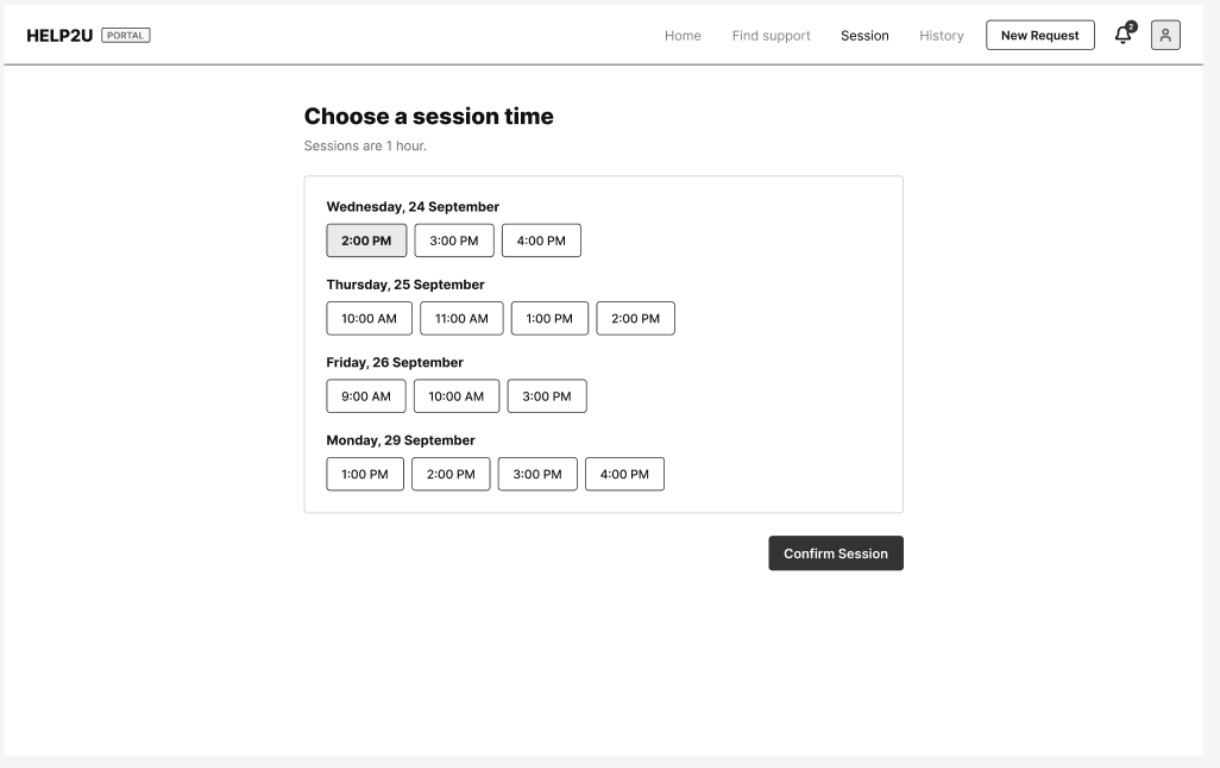


Figure 14: Screenshot of the Wireframe: Session Time Selection Page

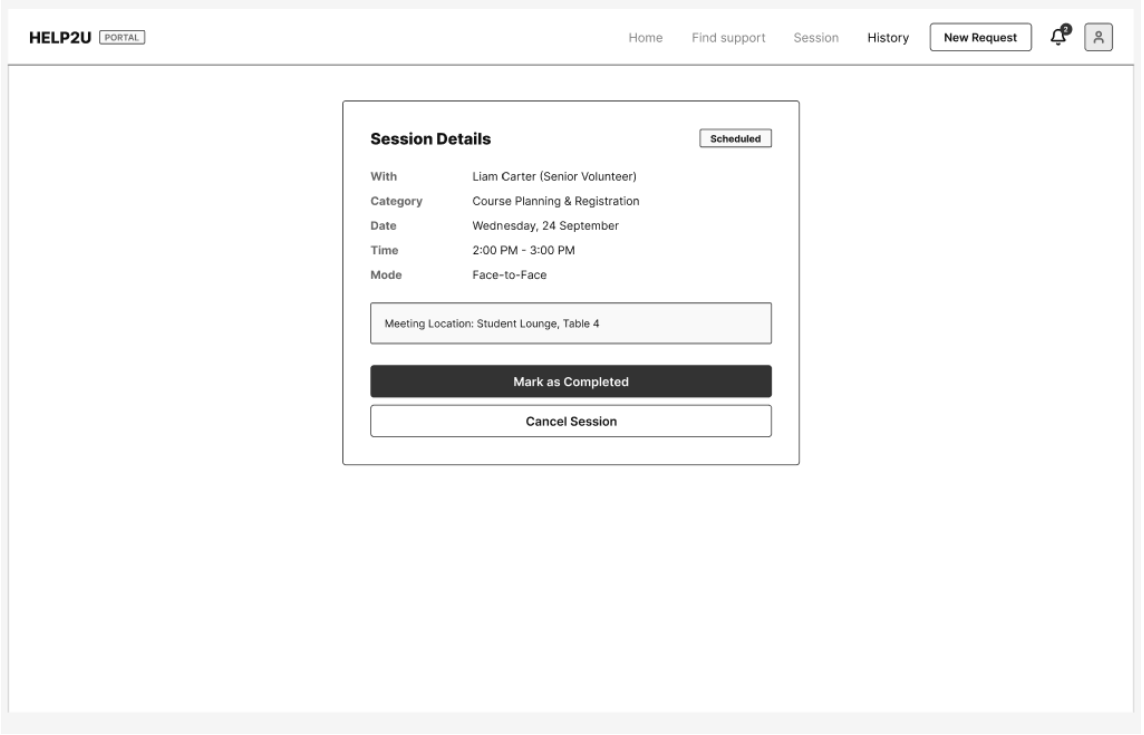


Figure 15: Screenshot of the Wireframe: Session Detail Page

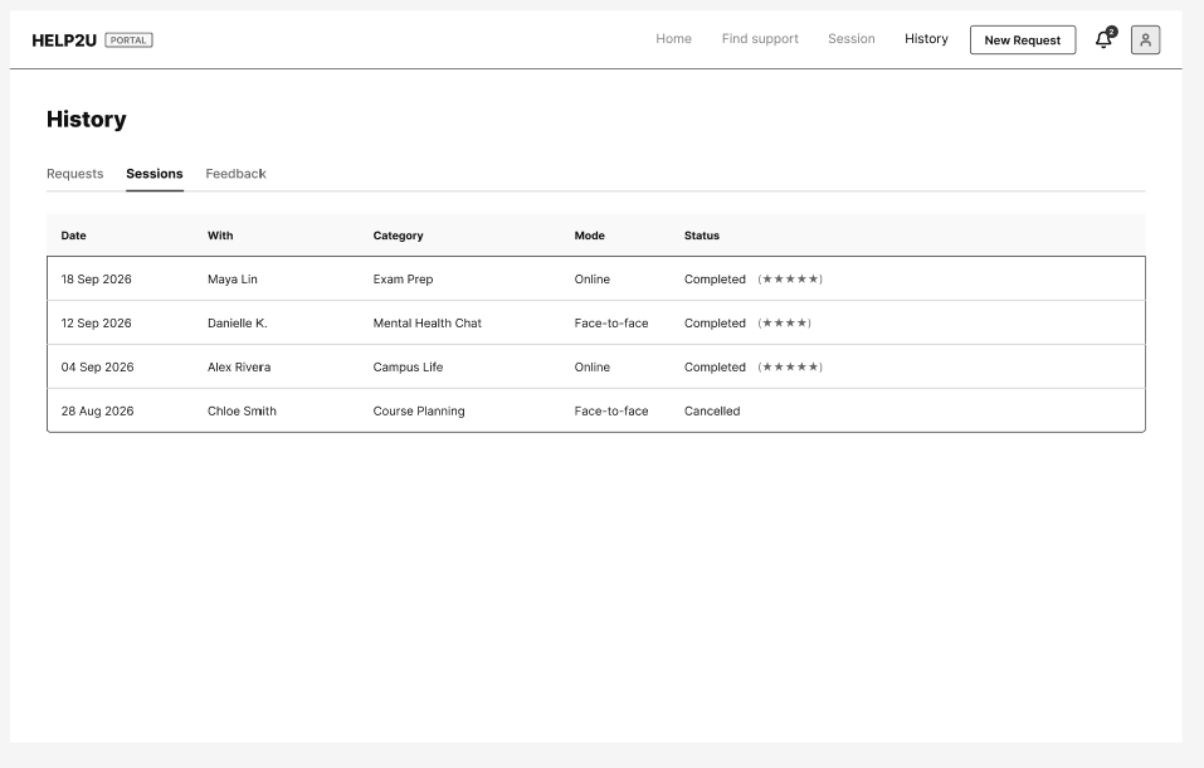


Figure 16: Screenshot of the Wireframe: History Page

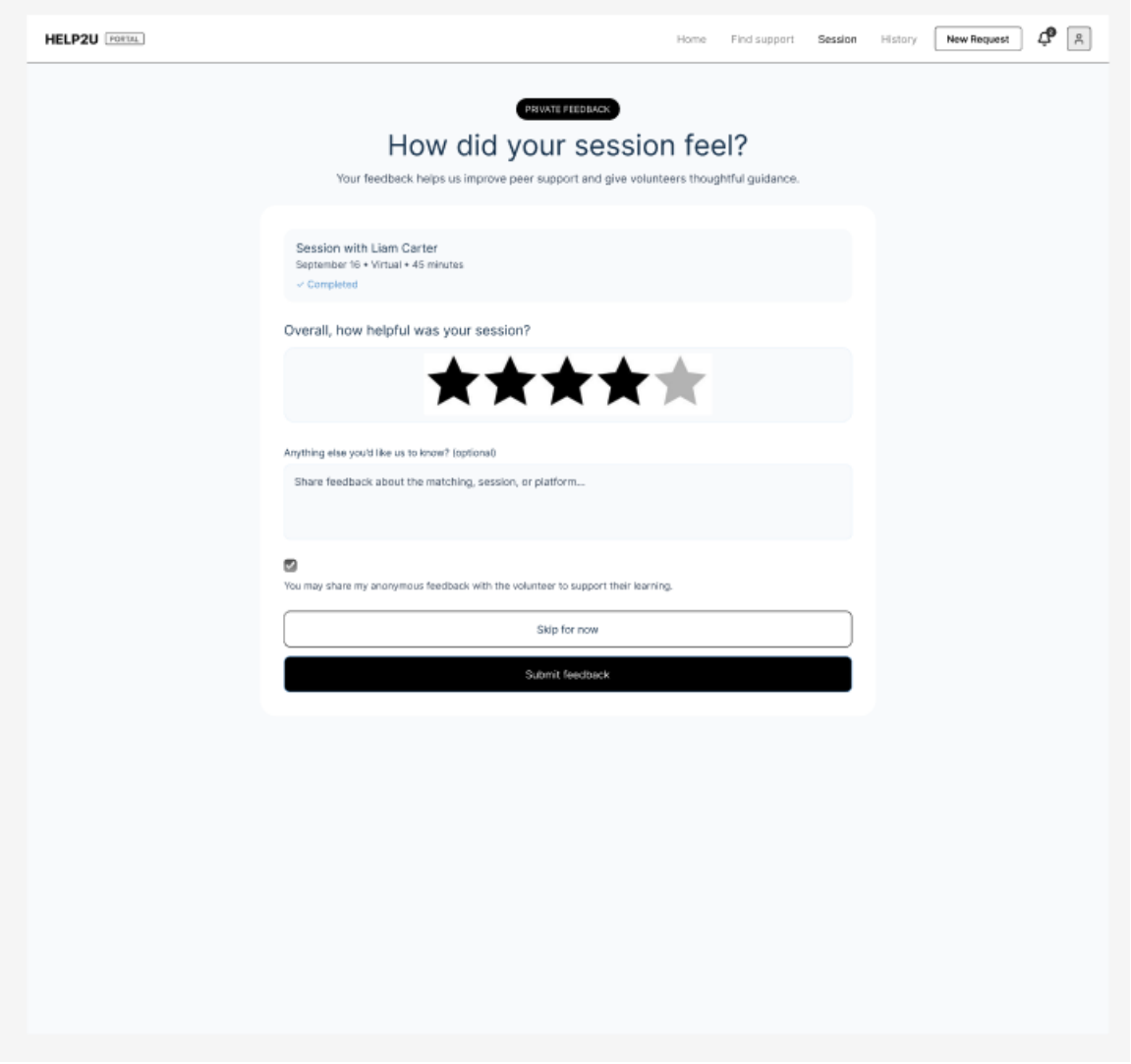


Figure 17: Screenshot of the Wireframe: Feedback Page

4.0 Marking Rubric

B. Requirements									
CLO2	Requirements definition [5]								
	Functional Requirements	3	Requirements are missing, vague or irrelevant.	Basic requirements identified but lack clarity/detail.	Clear and relevant requirements covering major system functions.	Comprehensive, precise and testable requirements clearly reflecting stakeholder needs.			
	Non-func. Requirements	2	NFRs are missing or inappropriate.	Basic NFRs identified but lack measurable criteria.	Relevant NFRs with reasonable criteria identified.	Specific, measurable and relevant NFRs addressing appropriate quality attributes.			
CLO2	Requirements specification [2+3 +5 + 5 = 15]						Student 1	Student 2	Student 3
	Epic	2	Missing, unclear or unrelated.	Relevant but broad or unclear.	Clear and relevant to the use case.	Clearly defines the user/business value and is strongly aligned with the use case.			
	User Stories	3	Missing or poorly written.	Basic stories provided but some lack clarity.	Clear, user-centred stories addressing relevant needs.	Well-written, appropriately scoped and clearly expressing user value.			
	AC	5	Missing, vague or not testable.	Basic criteria but some are ambiguous/incomplete.	Clear and testable criteria covering expected behaviour.	Specific, comprehensive and independently testable criteria.			
	Tasks	5	Missing or unrelated to stories.	Basic tasks identified but lack detail.	Relevant tasks logically derived from stories.	Detailed, actionable and logically sequenced implementation tasks.			
C. Design	Design [20]								
CLO2	Architectural Design	3	Missing or inappropriate architecture.	Basic architecture with limited explanation.	Appropriate architecture representing major components.	Well-structured and clearly justified architecture.			

	Site Map	3	Missing/incomplete or confusing navigation.	Basic pages identified but navigation has gaps.	Clear and logical navigation structure.	Comprehensive navigation hierarchy aligned with requirements.			
	Data Design[2] Data Dictionary[3]	5	Data requirements missing or inappropriate. Incomplete or largely inaccurate; key data elements are missing.	Basic data elements identified but relationships unclear. Basic data elements identified but definitions or details are incomplete.	Appropriate data elements and relationships identified. Relevant data elements are clearly defined with appropriate attributes and details.	Comprehensive data design clearly derived from requirements. Comprehensive and accurate data dictionary with clear definitions, data types, constraints and relationships aligned with the requirements			
	Wire frames [9]	9	Missing/incomplete or poorly aligned with requirements.	Basic screens provided but lack detail/consistency.	Clear and consistent screens covering main functions.	Comprehensive, consistent and user-centred wireframes demonstrating functionality and navigation.			
CLO3	Documentation [10]								
	Format Consistency	10	Major formatting inconsistencies.	Generally acceptable with several inconsistencies.	Consistent with minor issues.	Highly consistent and professionally formatted.			
	Spelling/Grammar		Frequent errors affect readability.	Some errors but generally understandable.	Few errors.	Virtually error-free and professionally written.			
	Professionalism		Poor presentation and inappropriate terminology.	Adequate presentation.	Professional presentation and terminology.	Highly polished and professional documentation.			
	TOTAL	**50**				**Sub-total (/50 marks)**	0	0	0
							Student 1	Student 2	Student 3